

| STAT214 Chapter 12 Decision Trees and the Random Forest Algorithm

| 1 Decision Trees

| 1.1 Decision Trees

- A binary decision tree consists of a tree-like sequence of binary questions involving the predictive features
- Starting with the entire training dataset at the top of the decision tree, these questions sequentially split the training data into smaller and smaller subsets, and a response prediction can be computed for each terminal leaf node based on the responses of the subset of training data points that it contains.
- the most powerful property of a decision tree lies in its interpretability and ease of use

| 2 The Classification and Regression Trees Algorithm

| 2.1 Choosing the Splits

| 2.1.1 The Variance-Splitting Measure for Continuous Responses

The CART algorithm chooses the split for which the responses in each child node will have minimum variance.

Computing this variance measure for a split involves computing a *weighted* combination of the variance across the left and right child nodes:

$$\text{Variance measure for split} = \frac{n_{\text{left}}}{n_{\text{parent}}} \text{Var}_{\text{left}} + \frac{n_{\text{right}}}{n_{\text{parent}}} \text{Var}_{\text{right}}$$

| 2.1.2 The Gini Impurity Splitting Measure for Binary Responses

The Gini impurity of a set of observations is defined as the probability that two randomly chosen observations (where we are assuming that the two observations are sampled *with replacement*) from the node are in different classes

$$\begin{aligned} \text{Gini impurity} &= \mathbb{P}(\text{two random obs in different classes}) \\ &= 1 - (\mathbb{P}(\text{both pos class}) + \mathbb{P}(\text{both neg class})) \\ &= 1 - p_1^2 - p_0^2 \end{aligned}$$

where

$$\begin{aligned} p_1 &= \mathbb{P}(\text{a randomly selected observation is in the positive class}) \\ p_0 &= \mathbb{P}(\text{a randomly selected observation is in the negative class}) \end{aligned}$$

Remark

Minimizing the continuous response variance measure in the context of binary responses is the same as minimizing the Gini impurity.

We need to combine the left- and right-child Gini impurity values

$$\text{Gini impurity for split} = \frac{n_{\text{left}}}{n_{\text{parent}}} \text{Gini}_{\text{left}} + \frac{n_{\text{right}}}{n_{\text{parent}}} \text{Gini}_{\text{right}}$$

| 2.2 Tuning the CART Hyperparameters

The CART algorithm has two main hyperparameters:

- **Maximum depth**: The maximum depth of the tree (i.e., the maximum number of sequential splits allowed). The default in the “rpart” R package is 30, and this hyperparameter is called `maxdepth`. For the `tree.DecisionTreeClassifier()` class from the Python scikit-learn library, this hyperparameter is called `max_depth` and there is no default maximum.

- **Minimum node size:** The minimum number of observations required to split a parent node. The default in the “rpart” R package is 20, and the relevant hyperparameter is called `minsplit`. For the `tree.DecisionTreeClassifier()` class from the Python scikit-learn library, this hyperparameter is called `min_samples_split` and the default is 2.

You can tune these hyperparameters using cross-validation (CV), but in practice, we often find that the default values (in R) work quite well.

| 2.3 Generating Predictions

Binary response problems

There are two types of response predictions:

- **Positive class probability prediction:** The predicted positive class probability is the proportion of the *training observations* in the leaf node that are in the positive class.
- **Binary class prediction:** A binary prediction can be computed using the “majority” class (the class that makes up at least 50 percent of the observations) of the *training observations* in the leaf node. For problems with class imbalance, it often makes more sense to instead use a threshold equal to the proportion of training set observations in the positive class (or tuned using CV).

Continuous response problems

Can be computed using the average response of the *training observations* in the leaf node.

| 2.4 A Summary of the CART Algorithm

Start with a top-level parent node that contains all the training data observations. Then:

1. Conduct a **greedy search** for the “best” split of the parent node by
 1. Identifying a range of reasonable possible splits each consisting of a binary question involving a predictor variable and continuous threshold or categorical level (such as `exit_rates < 0.05` ; `exit_rates < 0.024` ; `product_type_duration < 300` ; `month = "December"`).
 2. For each potential split option identified in the previous step, compute the variance of the responses (for continuous responses) or Gini impurity (for binary responses) in each of the resulting child nodes.
 3. Identify which of the potential splits leads to the lowest variance measure (for continuous responses) or Gini impurity measure (for binary responses) of the child nodes. This is the “best” split.
2. Implement the best split identified in the previous step to create two child nodes.
3. For each resulting child node that does not meet a prespecified stopping criteria, repeat steps 1 and 2 (i.e., each child node is now the parent node and will be split into subsequent child nodes). If a child node meets the prespecified stopping criteria do not split the child node. A child node that is not split further is called a **“leaf node”** or **“terminal node”**.
4. Continue computing sequential splits until all child nodes meet the prespecified stopping criteria.

To generate predictions for new observations, identify the relevant leaf node for the new observation and compute the average response of the training observations in the leaf node (for continuous responses) or the proportion of positive class training observations in the leaf node (for binary responses).

| 3 The Random Forest Algorithm

| 3.1 The Random Forest Algorithm

The Random Forest (RF) algorithm is an **ensemble** algorithm that combines the predictions from many decision trees. Randomness is introduced into each tree by

- Training each tree using a different random bootstrap sample of the training data.
- Considering a different random subset of the predictor variables for each node split.

For binary response problems, a class probability prediction can be computed based on the average probability prediction from the individual trees. Binary class response predictions are computed either using a majority vote of the individual tree binary predictions or by applying a threshold to the class probability predictions.

A continuous response prediction can be computed by averaging the predicted responses across the individual trees.

| 3.2 Tuning the RF Hyperparameters

The RF algorithm hyperparameters include

- **Number of variables to try:** The number of variables randomly selected for consideration at each split. The relevant hyperparameter in the “ranger” R package is `mtry` and the default is the (rounded down) square root of the total number of variables. The relevant hyperparameter for the `ensemble.RandomForestClassifier()` class from the scikit-learn library in Python is `max_features` (the default is the same as in R).
- **Maximum depth:** The maximum depth of the tree (i.e., the maximum number of sequential splits allowed). The maximum depth can be thought of as a *regularization* hyperparameter since a shallower tree with fewer splits is considered simpler. The “ranger” hyperparameter is `max.depth` and the hyperparameter for the `ensemble.RandomForestClassifier()` class from the scikit-learn library in Python is `max_depth`. Their default value for both implementations is unlimited depth.
- **Minimum node size:** The minimum number of observations in a node after which no more splits will occur. The “ranger” default is 1 for binary predictions, 10 for class probability predictions, and 5 for continuous response predictions. The relevant hyperparameter in the `ranger()` function in R is `min.node.size`. The relevant hyperparameter for the `ensemble.RandomForestClassifier()` class from the scikit-learn library in Python is `min_samples_split`, which has a default value of 2.
- **Number of trees:** The number of trees in the forest. The “ranger” hyperparameter is `num.trees`, which has a default of 500. The relevant hyperparameter for the `ensemble.RandomForestClassifier()` class from the scikit-learn library in Python is `n_estimators`, which has a default of 100.

These can be tuned using CV, but the default hyperparameter values typically work reasonably well across both the R and Python implementations.

| 4 Random Forest Feature Importance Measures

| 4.1 The Permutation Feature Importance Score

- The **permutation feature importance score** for a single feature involves identifying how much the mean squared error (MSE; for continuous responses) or prediction accuracy (for binary responses) decreases when retraining the RF algorithm using a permuted (a permutation is a type of perturbation) version of together with the original unpermuted versions of the other predictive features.
- it is more common to compute the **out of bag (OOB)** prediction error when evaluating the permutation feature importance. Since each tree in the RF ensemble is trained using a bootstrap sample of the training data, for each training data point, there will generally be some random subset of trees for which the data point was not involved in training. The **OOB prediction error** corresponds to the prediction error for each training data point based only on the trees whose training set did *not* include the data point.

| 4.2 The Gini Impurity Feature Importance Score

- The greater the reduction in impurity/variance from parent to child node, the better the split.
- the **Gini impurity feature importance score** for feature x can be computed by aggregating the decreases in Gini impurity (binary) or variance (continuous) across each split involving feature x across all the trees in the forest. This Gini impurity-based feature importance score is sometimes referred to as the *mean decrease in impurity (MDI)*.

| 5 PCS Evaluation of the CART and RF Algorithms

| 5.1 Predictability

On the *full* corresponding validation dataset, evaluate

- confusion matrix
- accuracy
- true positive rate
- true negative rate

- AUC
- ROC curve
- the distribution of predictions for each class

| 5.2 Stability

Prediction stability plots